FICHE TUTEUR · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 2

La base de données MySQL

Durée : 5 jours SQL Migrations Eloquent Relations Enums PHP

Intentions pédagogiques

Deux compétences en parallèle : **SQL brut** (indispensable même avec un ORM) et **Eloquent**. Faites systématiquement le lien : « cette ligne Eloquent, quelle requête SQL produit-elle ? ». Un développeur qui ne sait pas ce que son ORM envoie à la base écrira des applications lentes.

Le choix `restrict` / `null` / `cascade` des clés étrangères est le moment fort de la semaine : c'est un **choix métier**, faites-le argumenter.

Déroulé conseillé

Jour	Activité
J1	Migrations `users` (role, telephone), `sites`, `equipements`. `SHOW CREATE TABLE` pour comparer.
J2	Migrations `liaisons`, `incidents`, `suivis`. Discussion sur les clés étrangères.
J3	Enums et modèles avec relations. Tests dans Tinker.
J4	Factories et seeder. `migrate:fresh --seed` doit tourner sans erreur.
J5	Exercices SQL, point hebdomadaire.

Corrigé : fichiers de référence

- Migrations : `database/migrations/2026_10_29_00000*_create_*.php`
- Enums : `app/Enums/*.php`
- Modèles : `app/Models/*.php`
- Seeder : `database/seeders/DatabaseSeeder.php`

Choix de suppression du corrigé :

Clé étrangère	Choix	Justification
`equipements.site_id`	restrict	On ne perd pas un inventaire par erreur
`liaisons.site_a_id / site_b_id`	restrict	Idem
`incidents.site_id`	restrict	L'historique des incidents doit rester
`incidents.equipement_id / liaison_id`	null	L'équipement peut être décommissionné, l'incident reste
`incidents.assigne_a_id`	null	Un technicien qui part ne supprime pas ses incidents
`incidents.declare_par_id`	restrict	Traçabilité : on sait toujours qui a déclaré
`suivis.incident_id`	cascade	Un suivi n'a pas de sens sans son incident

Génération de la référence (bonus) :

```
protected static function booted(): void
{
    static::created(function (Incident $incident) {
        $incident->reference = sprintf('INC-%s-%05d', $incident->debut_at->format('Y'),
$incident->id);
        $incident->saveQuietly();
    });
}
```

Pourquoi après l'insertion ? L'id n'est connu qu'une fois la ligne insérée (AUTO_INCREMENT). Calculer « le nombre d'incidents + 1 » avant l'insertion crée des doublons si deux techniciens déclarent en même temps.

> **Piège du seeder** — Le seeder par défaut utilise le trait `WithoutModelEvents`, qui désactive l'événement `created` : toutes les références restent vides. Il faut retirer ce trait.

Corrigé des exercices SQL

```sql
-- 1. Sites triés par ville
SELECT * FROM sites ORDER BY ville;

-- 2. Équipements Cisco
SELECT hostname, ip_management FROM equipements WHERE constructeur = 'Cisco';

-- 3. Nombre d'équipements par type
SELECT type, COUNT(*) AS total FROM equipements GROUP BY type ORDER BY total DESC;

-- 4. Équipements avec le nom de leur site
SELECT e.hostname, e.type, s.nom AS site
FROM equipements e
JOIN sites s ON s.id = e.site_id;

-- 5. Liaisons avec leurs deux extrémités (double JOIN avec alias)
SELECT l.code, sa.code AS site_a, sb.code AS site_b, l.capacite
FROM liaisons l
JOIN sites sa ON sa.id = l.site_a_id
JOIN sites sb ON sb.id = l.site_b_id;

-- 6. Suppression refusée
DELETE FROM sites WHERE code = 'BKO-DC1';
-- ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails
```

Erreurs fréquentes

Erreur	Symptôme	Piste
Ordre des migrations	`Failed to open the referenced table 'sites'`	Le nom de fichier (horodatage) fixe l'ordre : `sites` avant `equipements`
`$fillable` oublié	`Add [code] to fillable property`	Expliquer le mass assignment
Relation mal nommée	`$liaison->siteA` renvoie `null`	Préciser la clé : `belongsTo(Site::class, 'site_a_id')`
Enum non casté	`$incident->severite->label()` → « Call to a member function on string »	Ajouter le cast dans `casts()`
Références vides	`reference` NULL partout	Trait `WithoutModelEvents` dans le seeder
`push()` modifie la collection d'origine	L'admin se retrouve dans la liste des techniciens	Utiliser `concat()` qui renvoie une nouvelle collection

Questions pour le point hebdomadaire

1. **Pourquoi `cascade` sur `suivis` mais `restrict` sur `equipements.site_id` ?** *Attendu :* un suivi n'existe pas sans incident ; un site avec des équipements actifs ne doit pas disparaître par erreur.

2. **Quelle requête SQL produit `Site::withCount('equipements')->get()` ?** *Attendu :* un `SELECT` avec une sous-requête `(SELECT COUNT(*) FROM equipements WHERE sites.id = equipements.site_id) AS equipements_count`.
3. **Pourquoi utiliser un enum plutôt qu'une simple chaîne ?** *Attendu :* liste fermée, fautes de frappe impossibles, libellés et couleurs centralisés, autocomplétion dans l'éditeur.
4. **À quoi sert un index ? Pourquoi ne pas indexer toutes les colonnes ?** *Attendu :* accélère les recherches ; mais ralentit les écritures et occupe de la place.
5. **Différence entre une factory et un seeder ?** *Attendu :* la factory sait fabriquer un modèle (données aléatoires), le seeder décide quoi créer pour la démo.

Grille d'évaluation (sur 20)

Critère	Points
Migrations complètes, types et contraintes justes	4
Clés étrangères avec comportements justifiés	3
Modèles, `#[Fillable]`, toutes les relations fonctionnelles	4
Enums avec `label()` / `color()` et casts	2
Seeder réaliste, `migrate:fresh --seed` sans erreur	3
Exercices SQL (6 requêtes justes)	4

Adapter le rythme

- **En avance** : lui faire écrire le scope `scopeActifs` et l'événement de génération de référence.
- **En difficulté** : fournir la migration `incidents` et l'enum `Severite` comme modèles ; il écrit les autres par analogie. Les exercices SQL 1 à 4 suffisent.